

# CND: Detecting Functionally Irrelevant Code Changes For Improving Regression Testing

1<sup>st</sup> Xin Zheng

*School of Computer Science and Technology* *School of Computer Science and Technology* *Sichuan Bangchen Information*  
*Southwest University of Science* *Southwest University of Science* *Technology Co., Ltd.*  
*and Technology* *and Technology* *Mianyang, China*  
Mianyang, China  
1311326350@qq.com

2<sup>nd</sup> Ya Pan\*

*Southwest University of Science*  
*and Technology*  
Mianyang, China  
panya@swust.edu.cn

3<sup>rd</sup> Xu Luo

*Technology Co., Ltd.*  
Mianyang, China  
gugia@qq.com

**Abstract**—Software regression testing is a crucial phase for maintaining software quality. To optimize the cost and efficiency of regression testing, selective test case generation and selection have become essential. Many studies have explored methods for reducing the number of test cases. However, during code changes such as bug fixes, feature additions, or optimizations, a portion of the changes may be functionally irrelevant, referred to as change noise. These changes, although affecting code structure, do not impact the software's behavior, syntax, or semantics.

In this paper, we propose a pattern-matching-based approach (CND) to detect change noise. By leveraging techniques such as pattern matching, refactoring detection, and semantic code transformation analysis, our method aims to identify functionally irrelevant changes. Our approach explores 26 types of change noise and detects them effectively.

To evaluate our change noise detection method, we selected 10 projects from certain public datasets [4-6] and expanded our research dataset by generating new commits through code transformations. The empirical results demonstrate that our method can effectively detect 66.34% of change noise. Additionally, we validated the effectiveness of detecting and removing change noise-related methods on optimizing regression test case selection in a project used for a web vulnerability testing range by a certain company. When applied to the regression test case selection for the project, this reduction in the number of methods associated with change noise indirectly decreases the number of test cases selection and generation, thereby improving regression testing efficiency.

**Index Terms**—information extraction, regression test, code change

## I. INTRODUCTION

Software regression testing is an crucial stage of agile development. Regression testing can generally be divided into full regression and partial regression. In smaller projects, the cost of full regression testing is usually acceptable; however, in larger projects, full regression can become cumbersome. To save testing costs and improve testing efficiency, partial regression is used in more scenarios, that is, regression test selection (RTS) [7]. Regression test selection aims to reduce regression testing costs by only selecting and running tests that may be affected by code changes. To date, various techniques for regression test case selection have been proposed, primarily including regression test selection [12-15,25-26], regression

test case reduction[22-24], and regression test case prioritization [16-19,21].

Benjawan et al. [1] executes existing test cases, records the detected faults and their severity levels in log files, and then prioritizes test cases with higher fault severity levels for subsequent regression testing. Hu et al. [2] introduce identification attributes for test cases. For instance, the number of defect detection indicates the total number of failures the test case has experienced during historical executions. The importance factor reflects the significance of the test case in relation to the current testing requirements. Additionally, the new and old function identifiers indicate whether the test case belongs to a newly added or deleted module. Based on the regression testing goals at different stages, this method automatically sorts the test cases according to these attributes and selects the test case set based on the sorting results. Zhou et al.[3] proposed a regression test selection method based on the probability of accidental correctness, which aims to enhance fault detection capabilities by removing test cases that may exhibit accidental correctness. In particular, Code change is one of the most critical triggers for regression testing.

However, prioritizing the selection of regression test cases affected by code changes may result in unnecessary case selection and testing execution for functionally irrelevant changes, thereby increasing testing costs. Kawrykow et al. [28] observed that significant changes to software artifacts are sometimes accompanied by many unnecessary modifications, such as textual differences caused by local variable refactoring or renaming refactoring. They developed a tool called DIFFCAT to detect unnecessary code differences in the revision history of software systems. They validated their approach on over 24,000 change sets collected from the change histories of seven long-lived open-source systems and found that 15.5% of the system's method updates were solely due to non-essential differences. The change noise studied in this paper is similar to unnecessary changes. We define change noise as textual differences in a change activity that do not affect functional changes, i.e., functionally irrelevant changes. These include code formatting perturbations, code refactoring, and semantically equivalent modifications. We believe that

such changes introduce unnecessary test case selections for regression testing, thereby increasing the testing burden to some extent, as they are not the focal points that take high attention in a change activity. Thus, detecting functionally irrelevant changes is crucial.

In this paper, we propose a pattern-matching approach to detect code change noise. We explore 26 types of change noise and employ pattern matching to identify functionally irrelevant changes. We argue that minor changes that do not affect original functionality can be heuristically ignored in regression testing, as they are essentially equivalent in syntax and semantics, yielding consistent testing results. Based on test dependency information and the granularity of program changes, **regression test selection can be categorized into various levels, with our study focusing on function-level change detection.**

Due to the limited number of change noise cases in existing public datasets, we selected project datasets used in prior studies, filtering ten projects as a baseline dataset, and employed semantic equivalence transformation rules to generate new code revision commits, thus constructing our change noise cases. Experimental results indicate that our method effectively detects 66.34% of change noise, and by reducing the number of functions associated with these changes, we indirectly enhance the efficiency of regression testing.

The main contributions of this paper are as follows:

- We summarize and explore 26 types of change noise and construct a dataset of change noise commits based on semantic equivalence transformation.
- We manually conduct functionally irrelevant changes in a real project and validate our method. The experimental results show that our approach significantly reduces the selection and generation of test cases compared to not using the change noise detection method, improving regression testing efficiency.

The rest of the paper is organized as follows. Section II describes our approach. Section III details our datasets, evaluation metrics, and research questions. Section IV conducts experiments and analyzes results. Section V introduces related work. Section VI summarizes the paper.

## II. METHOD

In this section, we introduce the details of our approach in this paper. We first summarize the types of change noise we are currently exploring and describe how we constructed the dataset required for our study. Then, we leverage our change noise detection method to identify noise within the changes and associate it with functions. **Finally, we describe the performance comparison between the regression test case reduction using our proposed noise detection method and that without it.** Fig. 1 shows the overview framework of our approach, which is mainly divided into three parts and we will introduce it below.

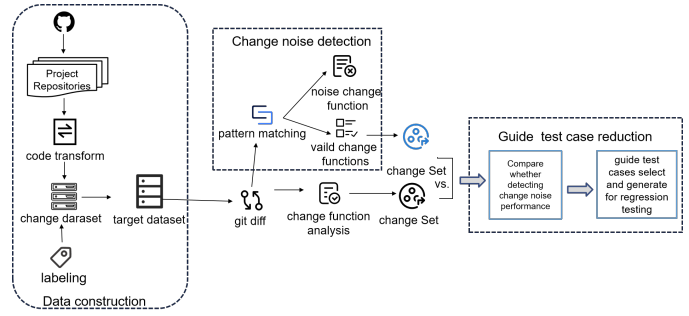


Fig. 1. The overview framework of our approach.

### A. Dataset construction

The first part is constructing the change noise commits dataset. Ideally, we would assess the reduction in regression test case selection by applying the heuristic change noise detection method proposed in this paper to the historical change commits of multiple open-source projects. We would then confirm with developers whether the detected change noise does not lead to essential functional changes. However, our research indicates that it is challenging to find perfectly matching revision cases for the types of change noise we are exploring, given the actual scenarios leading to revisions in the extensive historical commits of existing public datasets. Moreover, conducting such experiments with real developers is very difficult. Therefore, we selected 10 Java projects from public datasets used in studies [4-6], prioritizing projects included in these datasets and focusing on Java Web-related projects with a code size of over 20K lines.

These projects vary in code size and are open-source on GitHub. We then performed random transformations on the code using 17 semantic equivalence rules proposed in study [8] to generate code change commits. Specifically, the transformation rules require the target code to contain convertible features, meaning these rules cannot cover every project entirely. Therefore, we complemented these with manual revisions to ensure that each revision includes at least one instance of change noise.

In addition, to verify the effectiveness of regression test case selection after detecting and removing functions corresponding to noise changes, it is necessary to associate the test cases (or test suites) in the project with the functions being tested, in order to simulate the performance of our method in regression test case selection. However, our investigation found that few open-source projects have standardized and comprehensive public test cases, and the number of unit tests is often insufficient to cover all functionalities in the project. This is because our ideal scenario is that each function has a corresponding test case that can trigger every branch path.

Therefore, this paper assumes that each function is associated with at least one test case. We adopted a method of calculating the number of branch paths in a function to simulate the number of test cases associated with each function in the project.

### B. Change noise detection

The second part is change noise detection. Our goal is to detect the 26 types of change noise we have defined by leveraging heuristic strategies, combined with pattern matching techniques, refactoring detection technologies, and mining of semantically equivalent change patterns. TABLE II shows our descriptions of these 26 types of change noise.

The study [8] proposed 17 transformation rules that preserve semantic and syntactic naturalness, which they used to enhance program data and validate in three major code tasks: method naming, code commenting, and code clone detection. In the previous section, we applied these transformation rules to perform equivalence transformations on the program, resulting in change commits. Since these transformations do not impact the original functionality, they align with our definition of change noise. Detecting change noise treats code changes as differential text, relying on heuristic rules and refactoring detection techniques to match patterns. We evaluate the change noise detection method using recall and precision metrics [9].

The heuristic strategy in this paper first customizes pattern matching for the common characteristics of each type of noise change as much as possible. For example, for the PP2AddAssignment type shown in TABLE II, we first use the Levenshtein distance to calculate whether the similarity between the pre- and post-change text reaches the threshold  $xi$ , which is set to 0.75. Then, we determine whether the pre-change difference lines (added or deleted) contain “++” or “-” to match the initial characteristics of this type. Next, we determine whether the other text characters before and after the change are consistent to ensure that there are no change operations outside this type. Finally, we further determine whether the post-change text (added or deleted) contains “+=” or “-=” to confirm whether the change is a noise change. Listing 1 shows an case of the PP2AddAssignment change noise type.

```

1 @@ -66,7 +66,7 @@ public class MethodParameters {
2     MethodParameter parameter = new
3         AnnotationNamingMethodParameter(method, i,
4         namingAnnotation);
5 -     i++;
6 +     i+=1;

```

Listing 1. Change Case of PP2AddAssignment.

When the designed patterns cannot to match the corresponding type of noise change, we further employ pattern-based refactoring detection techniques and semantic equivalent transformation change pattern mining techniques for identification. For refactoring detection, we adopted the AST-based pattern matching algorithm proposed in [20], which can identify up to 59 types of Java refactorings from commit history. Rminer, an excellent refactoring detection tool, achieves an overall precision of 99.7% and a recall of 94.2% in recognizing refactoring operations [14]. The semantic-equivalent change pattern mining technique [27] is a novel graph-based mining method that detects previously unknown repetitive changes by mining fine-grained semantic code change patterns from

a large number of repositories. We manually constructed commit records of certain change noise as input and used this technique to mine fine-grained semantic code change patterns from those commits. These patterns were then used to detect certain change noise. Listing 2 shows an case of the For2While change noise type.

```

1 - for (; classMappingParameters.hasNext(); ) {
2 + while (classMappingParameters.hasNext()) {
3     values.put(names.next(), encodePath(
4         classMappingParameters.next()));

```

Listing 2. Change Case of For2While.

### C. Guide test case reduction

The third part is guide test case reduction. We compared the effectiveness of regression test case selection reduction using and not using the change noise detection method, as both approaches conduct an analysis of the set of functions affected by the changes (change impact analysis). Our point is the functions impacted by the changes, including those on the call paths of these functions.

Regression testing is performed based on the test case set associated with the non-selected functions, which reduces the selection of regression test cases and guides the generation and design of test cases to focus solely on the effective change functions (excluding change noise). By removing the regression test cases associated with the set of functions related to change noise, we narrow the focus to selecting and generating test cases specifically for these functions. In other words, our method reduces the number of test cases in this portion to decrease the time required for regression testing, thereby accelerating the regression testing process and improving its accuracy.

## III. EXPERIMENT

### A. Datasets

The datasets in this paper are derived from recent studies related to code repair, test case selection, and code defect detection [4-6], encompassing a total of 206 project repositories. We selected several projects of varying scales, including large, medium, and small-sized projects. The large-scale projects consist of multiple functional components, each capable of running independently. These projects have a commit history spanning several years and have received new commits in the past five years. This paper presents experiments on 10 of these projects, as detailed in TABLE I.

In this study, we applied the 17 semantic equivalence transformation rules proposed by Yu et al. [18] to the 10 selected projects using a random approach to generate change commits. To meet our research objectives, we categorized the transformations into six types related to code refactoring and three types related to formatting noise. These types were manually revised to produce change commits. Ultimately, our research dataset included 1,744 commits.

We employed a method to estimate the number of branch paths in functions, which we used to simulate the number

TABLE I  
DATA SET DETAILS

| Project           | Commits | Start SHA | size   | lines total | java files |
|-------------------|---------|-----------|--------|-------------|------------|
| spring-hateoas    | 95      | 2ba2670   | Small  | 20122       | 161        |
| helios            | 54      | 0b28559   | Middle | 70820       | 493        |
| RuoYi             | 54      | adb2284   | Middle | 35557       | 271        |
| WebGoat           | 54      | 61720b2   | Middle | 27780       | 355        |
| symja-parser      | 102     | 689f67a   | Small  | 11459       | 75         |
| Scribe-Java       | 54      | bfb536    | Small  | 21102       | 315        |
| netty             | 55      | 432fdbf   | Large  | 515625      | 2912       |
| mybatis           | 56      | 1e19afd   | Large  | 104735      | 1284       |
| spring-data-neo4j | 56      | ccc9329   | Middle | 58878       | 734        |
| spring-data-jpa   | 57      | 3483e16   | Middle | 80980       | 527        |

of test cases for each function in the project. The simulation is detailed in Algorithm 1. We assume that each method is connected to at least one test case and avoid detailed calculations of function calls within blocks to prevent path explosion due to recursion.

**Algorithm 1** Function Branch Path Count Algorithm

**Input:**

*function* : The function block to be analyzed;

**Output:**

*pathCount* : The total number of execution paths calculated;

```

1: pathCount  $\leftarrow$  1
2: for s in function do
3:   if s is IfStatement then
4:     pathCount  $\leftarrow$  pathCount + 1
5:   else if s is SwitchStatement then
6:     caseCount  $\leftarrow$  countCases(s)
7:     pathCount  $\leftarrow$  pathCount + (caseCount - 1)
8:   else if s is LoopStatement then
9:     pathCount  $\leftarrow$  pathCount + 1
10:  else if s is TryStatement then
11:    pathCount  $\leftarrow$  pathCount + 1
12:  else if s is MethodInvocation then
13:    continue
14:  end if
15: end for
16: return pathCount;
```

**B. Evaluation metrics**

To evaluate the performance of our change noise detection, we refer to the commonly used evaluation metric recently studies [10].

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (1)$$

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (2)$$

where recall is the percentage of truly related change noise that are correctly detected by the change noise detection algorithm. Precision represents the percentage of true change noise identified by the change noise detection algorithm relative to the total number of change noise.

**C. Research questions**

In this paper, we propose the following research questions (RQs) to evaluate our method.

- RQ1: How effective is the change noise detection?
- RQ2: What is the impact of removing detected change noise on the effectiveness of regression test case selection reduction?

RQ1 evaluates the performance of our change noise detection. RQ2 explores the effect of eliminating recognized change noise on the reduction of regression test case selection.

**D. Experimental Settings**

To ensure sufficient coverage of various transformation rules across different projects, we require that the target code contains convertible features. As a result, the final dataset is based on random transformations, with the number of transformations for each project set to 100. For multi-module projects, transformations are applied at the module level, with each module undergoing 100 random transformations.

In our approach, the transformation rules is implemented using the Eclipse's JDT [11] library.

IV. EXPERIMENTAL RESULTS

In this section, we present and explain the experimental results to answer two RQs.

**A. RQ1: How effective is the change noise detection?**

**Design.** We first execute the dataset construction process introduced in Section II-A to obtain the target dataset for our study, and we label each commit in the dataset with the corresponding change noise type. Then, we apply the change noise detection algorithm described in Section II-B to detect and match functionally irrelevant changes in the dataset. Finally, we evaluate the performance of our approach using recall and precision metrics as described in Section III-B. During the dataset construction process, we count the number of successful transformations for each semantic-equivalent transformation rule in the dataset, which helps reflect the transformation frequency across projects of varying sizes, as shown in TABLE III.

**Results.** TABLE III presents the experimental results of the change noise detection algorithm across 10 projects. The numbers in bold indicate the performance of correctly detected change noise types. TABLE IV shows the detection performance for the 26 types of change noise we analyzed. The bold numbers represent the change noise types that were detected with higher accuracy. The experimental results indicate that our change noise detection algorithm achieved a precision of 66.34%, meaning our approach was able to detect 66.34% of the change noise. Additionally, the detection performance was better for noise types related to code refactoring and format disturbances. The recall rate was 95.22%, meaning our method successfully detected 95.22% of the actual change noise.

TABLE II  
THE DETAILED INFORMATION OF CHANGE NOISE.

| Category                        | Noise Type                   | Description                                                                                                                                   |
|---------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Code equivalence transformation | ForEach-For-While            | Mutually replace semantic-equivalent loop statements.                                                                                         |
|                                 | if-else-switch               | Mutually replace if-else and switch statement blocks.                                                                                         |
|                                 | VarDeclDividing              | Divide the composite declaration statement into separated declaration statements.                                                             |
|                                 | LoopIfContinue-Else          | Convert IfContinue loop statement blocks to equivalent IfElse statement blocks                                                                |
|                                 | PrePostFixExpressionDividing | Split the prefix or postfix expression into two separate expressions, and add corresponding variable declarations to store these expressions. |
|                                 | ReverseIfElse                | Swap the two code blocks in the if statement and the corresponding else statement.                                                            |
|                                 | IfDividing                   | Divide the if statement with compound conditions into two nested if statements.                                                               |
|                                 | VarDeclarationMerging        | Combine the declaration statements into a single compound declaration statement.                                                              |
|                                 | SwapEqualSides               | Swap the instances being compared on either side of the equals method.                                                                        |
|                                 | SwapStringEqual              | Swap the strings being compared on either side of the equals method.                                                                          |
|                                 | InfixExpressionDividing      | Divide a infix expression into two expressions, and add corresponding variable declarations to store these expressions.                       |
|                                 | ConditionalExp2SingleIf      | Change the conditional expression statement to a single if statement.                                                                         |
|                                 | SingleIf2ConditionalExp      | Convert the if statement to a conditional expression.                                                                                         |
|                                 | StringDividing               | Split the string literal into the concatenation of two string literals.                                                                       |
|                                 | StringMerging                | Merge two string literals into a single string literal.                                                                                       |
|                                 | PP2AddAssignment             | Change the assignment x++ into x+=1.                                                                                                          |
|                                 | StmtsOrderReRank             | Switch the places of two adjacent statements in a code block, where the former statement has no shared variable with the latter statement.    |
| Code Refactor                   | LocalVarRenaming             | Rename variables, attributes and fields.                                                                                                      |
|                                 | MethodRenaming               | Rename the method name.                                                                                                                       |
|                                 | ExtractMethod                | Extract the code snippet into a function.                                                                                                     |
|                                 | MoveParameters               | Move the declaration of variables and fields to the appropriate position within the current block.                                            |
|                                 | MoveMethod                   | Move the function block to a different position within the current class file.                                                                |
| Format Noise                    | ParamOrderReRank             | Reorder the function parameters and adjust the corresponding arguments in the calling locations accordingly.                                  |
|                                 | ADBlank                      | added and deleted blank line or line break.                                                                                                   |
|                                 | ADMCComment                  | added, deleted and modified comment contents.                                                                                                 |
|                                 | ADBlankSpace                 | added and deleted blank space or tab.                                                                                                         |

TABLE III  
THE RESULTS OF CHANGE NOISE DETECTION OF EACH PROJECT.

| Project           | Count of Transform commits | Count of TP |
|-------------------|----------------------------|-------------|
| spring-hateoas    | 102                        | 56          |
| helios            | 73                         | 30          |
| RuoYi             | 87                         | 51          |
| WebGoat           | 91                         | 67          |
| symja-parser      | 102                        | 69          |
| Scribe-Java       | 136                        | 85          |
| netty             | 766                        | 489         |
| mybatis           | 97                         | 39          |
| spring-data-neo4j | 262                        | 167         |
| spring-data-jpa   | 28                         | 20          |
| <b>Total</b>      | <b>1744</b>                | <b>1157</b> |

TABLE IV  
THE RESULTS OF CHANGE NOISE DETECTION OF EACH NOISE TYPE.

| Noise Type                   | Count of Transform commits | Count of TP | No. of FP |
|------------------------------|----------------------------|-------------|-----------|
| ForEach-For-While            | 128                        | 65          | 0         |
| if-else-switch               | 23                         | 14          | 0         |
| VarDeclDividing              | 56                         | 50          | 0         |
| LoopIfContinue-Else          | 10                         | 8           | 0         |
| PrePostFixExpressionDividing | 15                         | 0           | 0         |
| ReverseIfElse                | 222                        | 179         | 0         |
| IfDividing                   | 47                         | 0           | 0         |
| VarDeclarationMerging        | 72                         | 66          | 0         |
| SwapEqualSides               | 186                        | 151         | 35        |
| SwapStringEqual              | 45                         | 40          | 5         |
| InfixExpressionDividing      | 76                         | 63          | 0         |
| ConditionalExp2SingleIf      | 77                         | 0           | 0         |
| SingleIf2ConditionalExp      | 79                         | 0           | 0         |
| StringDividing               | 30                         | 1           | 0         |
| StringMerging                | 45                         | 3           | 0         |
| PP2AddAssignment             | 49                         | 38          | 11        |
| StmtsOrderReRank             | 152                        | 107         | 0         |
| LocalVarRenaming             | 212                        | 212         | 0         |
| MethodRenaming               | 52                         | 52          | 0         |
| ExtractMethod                | 16                         | 16          | 0         |
| MoveParameters               | 12                         | 12          | 0         |
| MoveMethod                   | 23                         | 23          | 0         |
| ParamOrderReRank             | 53                         | 50          | 0         |
| ADBlank                      | 20                         | 18          | 2         |
| ADMCComment                  | 24                         | 23          | 1         |
| ADBlankSpace                 | 20                         | 16          | 4         |
| <b>Total</b>                 | <b>1744</b>                | <b>1157</b> | <b>58</b> |

B. RQ2: What is the impact of removing detected change noise on the effectiveness of regression test case selection reduction?

**Design.** We applied the full process described in Section II-A to a real-world project, which consists of 185 code files and 14,935 lines of code. This project is designed for detecting web vulnerabilities. During the dataset construction phase, we first applied random transformations to the project using transformation rules, followed by manually introducing changes outside of the transformation rules. We constructed 10 commit cases, with these manual changes containing both change noise and non-change noise. Non-change noise refer to changes that fall outside the 26 noise types we summarized, or those related to functional changes.

In the change noise detection phase, we used a function branch path counting algorithm to pre-calculate the number of test cases associated with each function in the project. This allowed us to clearly observe the impact of removing detected noise functions on regression test case reduction. We then performed change impact analysis on the project with use and disuse change noise detection(CND) and compared the sets of

impacted functions identified in both processes.

**Results.** Using the function branch path counting algorithm, we calculated that the total number of test cases associated with all functions in the project was 842. TABLE V and TABLE VI show two cases of the reduction in regression test case selection using change noise detection versus without it. While in Case 2 fewer noise functions were detected compared to Case 1, the reduction in test cases was very similar. Upon manual analysis, we found that one noise function in Case 2 had a branch path count of 20, meaning it was associated with more test cases, whereas the noise function with the highest branch path count in Case 1 had only 6. Therefore, when the excluded noise functions are associated with more test cases,

our method becomes more effective. The experimental results demonstrate that removing noise functions is indeed effective in reducing the number of regression test cases, eliminating the need to select or generate test cases for functions that do not require regression testing.

TABLE V  
THE PERFORMANCE CASE 1 OF CHANGE NOISE DETECTION FOR REGRESSION TEST CASE SELECTION AND REDUCTION.

|            | Count of Change Function Set | Count of associate Test Cases |
|------------|------------------------------|-------------------------------|
| use CND    | 13                           | 113                           |
| disuse CND | 21                           | 155                           |

TABLE VI  
THE PERFORMANCE CASE 2 OF CHANGE NOISE DETECTION FOR REGRESSION TEST CASE SELECTION AND REDUCTION.

|            | Count of Change Function Set | Count of associate Test Cases |
|------------|------------------------------|-------------------------------|
| use CND    | 5                            | 39                            |
| disuse CND | 7                            | 67                            |

## V. RELATED WORK

There is a substantial body of work on change impact analysis, regression test selection, and test case reduction, but to the best of our knowledge, there has been no prior work focusing on reducing regression test cases by excluding those associated with changed functions.

**Code Commit Classification.** Analyzing software maintenance activities is crucial for ensuring cost-effective evolution and development. Some studies have analyzed software commit messages to extract fine-grained change patterns, such as refactoring operations [29], API migrations [30], and defect fixes [31]. These studies aim to understand the nature of changes and their impact on software systems. For example, Hindle et al. [32] used machine learning to classify changes into maintenance categories (corrective, adaptive, perfective) and additional categories like feature additions. Gharbi et al. [33] proposed a multi-label active learning method for commit classification, enabling multi-label commit classification. Code commit classification helps in understanding the intent behind code changes, improving the efficiency of subsequent maintenance and review activities. we aim to detect functionally irrelevant changes to enhance the efficiency of testing, maintenance, and code review.

**Change Impact Analysis.** Change Impact Analysis (CIA) is a specialized process in program comprehension that examines the ripple effects of code changes within a software system. Some studies have analyzed change behaviors by mining code change patterns. Janke et al. [23] proposed a method to automatically mine code change patterns from version control systems using frequent subgraphs, capturing contextual relationships between code edits. They emphasize the scalability and semantic richness of their approach. Huang et al. [9] introduced a method to identify related code changes

by extracting 21 coupling rules and 4 co-evolution types based on object-oriented programming paradigms, using machine learning to recognize code change relationships. Huang et al. [10] further proposed a method for cross-project CIA, mapping historical change patterns to current tasks, reflecting coupling dependencies between entities that frequently change together.

**Regression Test Selection.** Regression testing is essential for detecting software defects, and selecting regression test cases is a critical step. In recent years, the minimal test case set strategy has gained popularity, focusing on filtering previous test cases. Liu et al. [35] explored a method that combines filtering test cases based on design and execution results with new test case selection approaches, including exploratory testing. As software grows, so does the number of regression test cases, making it crucial in Agile development to extract test cases that are most likely to detect defects, thus shortening release cycles. Shantanu et al. [36] proposed a natural language processing-based approach to identify high-potential regression test cases by matching them with defect intent. Zhang et al. [7] introduced HyRTS, a hybrid regression test selection method that analyzes multiple granularities (e.g., basic blocks, methods, and files) to leverage the strengths of different traditional techniques, combining method-level and file-level RTS.

## VI. CONCLUSION

This paper proposes a pattern-based approach to detect functionally irrelevant code changes, defined as change noise. We identified 26 types of such changes and used heuristic pattern matching, refactoring detection, and semantic equivalence mining to recognize them. To evaluate our method, we expanded a public dataset with 1,774 new commits generated through code transformations. Experiments showed that our approach effectively detected 66.34% of change noise. Applying this method to a real-world project reduced the number of associated functions for regression test case selection, saving time and effort and enhancing regression testing efficiency.

However, our approach has limitations. Firstly, our pattern-based detection algorithm, tailored for Java, does not generalize well to other languages like C#, Python, or C++. Due to the unique syntax and semantics of each language, such as Python's dynamic typing and indentation-based code block structure, or C++'s templates and operator overloading, additional language-specific rules and patterns would need to be defined. This would involve analyzing the unique features of each language and developing tailored pattern-matching rules to identify functionally irrelevant changes. Secondly, detection results for some change noise types were less favorable than expected, likely due to the limitations of heuristic pattern matching. Future work could explore intelligent algorithms to improve detection accuracy. Additionally, we plan to extend our method to more projects and investigate additional types of functionally irrelevant changes.

## REFERENCES

- [1] Srisura, Benjawan, and Adtha Lawanna. "False test case selection: Improvement of regression testing approach." 2016 13th International



- Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON). IEEE, 2016.
- [2] Hu Peng, Chang Chaowen, Zhu Xianwei, et al. Regression testing optimization method for continuous integration [J]. *Application Research of Computers*, 2021, 38 (12): 3709-3714.
  - [3] Zhou XL, Zhao JH. Regression Test Selection Method Based on Coincidental Correctness Probability. *Journal of Software*, 2021, 32(7): 2103-2117(in Chinese). <http://www.jos.org.cn/1000-9825/6264.htm>
  - [4] Kui, L., Dongsun, K., Tegawende F., B., Shin, Y., & Yves Le, T. (2021) Mining Fix Patterns for FindBugs Violations, *IEEE Transactions on Software Engineering*, 47.1: 165-188.
  - [5] Shahbaz, M. (2023). An Approach for Test Impact Analysis on the Integration Level in Java Programs. In: Yang, X.S., Sherratt, R.S., Dey, N., Joshi, A. (eds) *Proceedings of Eighth International Congress on Information and Communication Technology*. ICICT 2023.
  - [6] Shi, A., Yung, T., Gyori, A., & Marinov, D. (2015, August). Comparing and combining test-suite reduction and regression test selection. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering* (pp. 237-247).
  - [7] Lingming, Z. (2018) Hybrid Regression Test Selection, *International Conference on Software Engineering*: 199-209. Yu, S., Wang, T., & Wang, J. (2022). Data augmentation by program transformation. *Journal of Systems and Software*, 190, 111304.
  - [8] Yu, S., Wang, T., & Wang, J. (2022). Data augmentation by program transformation. *Journal of Systems and Software*, 190, 111304.
  - [9] Huang, Y., Chen, X., Liu, Z., Luo, X., & Zheng, Z. (2017). Using discriminative feature in software entities for relevance identification of code changes. *Journal of Software: Evolution and Process*, 29(7), e1859.
  - [10] Huang, Y., Jiang, J., Luo, X., Chen, X., Zheng, Z., Jia, N., & Huang, G. (2021). Change-patterns mapping: A boosting way for change impact analysis. *IEEE Transactions on Software Engineering*, 48(7), 2376-2398. <https://github.com/eclipse-jdt>.
  - [12] J. Chen, Y. Bai, D. Hao, L. Zhang, L. Zhang, and B. Xie. How do assertions impact coverage-based test-suite reduction? In *International Conference on Software Testing, Verification and Validation*, pages 418-423, 2017.
  - [13] D. Di Nardo, N. Alshahwan, L. Briand, and Y. Labiche. Coverage-based regression test case selection, minimization and prioritization: A case study on an industrial system. *Software Testing, Verification and Reliability*, 25(4):371-396, 2015.
  - [14] A. Shi, A. Gyori, M. Gligoric, A. Zaytsev, and D. Marinov. Balancing trade-offs in test-suite reduction. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 246-256, 2014.
  - [15] Chenguang, Z., Owolabi, L., August, S., & Milos, G. (2019) A Framework for Checking Regression Test Selection Tools, *International Conference on Software Engineering*.
  - [16] D. Hao, L. Zhang, L. Zhang, G. Rothermel, and H. Mei. A unified test case prioritization approach. *Transactions on Software Engineering and Methodology*, 24(2):10:1-10:31, 2014.
  - [17] Y. Lu, Y. Lou, S. Cheng, L. Zhang, D. Hao, Y. Zhou, and L. Zhang. How does regression test prioritization perform in real-world software evolution? In *International Conference on Software Engineering*, pages 535-546, 2016.
  - [18] R. K. Saha, L. Zhang, S. Khurshid, and D. E. Perry. An information retrieval approach for regression test prioritization based on program changes. In *International Conference on Software Engineering*, volume 1, pages 268-279, 2015.
  - [19] K. Zhai, B. Jiang, and W. K. Chan. Prioritizing test cases for regression testing of location-based services: Metrics, techniques, and case study. *Transactions on Services Computing*, 7(1):54-67, 2014.
  - [20] N. Tsantalis, M. Mansouri, L. Eshkevari, D. Mazinanian, and D. Dig. "Accurate and efficient refactoring detection in commit history," in 2018 *IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 2018, pp. 483-494.
  - [21] K. Zhai, B. Jiang, and W. K. Chan. Prioritizing test cases for regression testing of location-based services: Metrics, techniques, and case study. *Transactions on Services Computing*, 7(1):54-67, 2014.
  - [22] M. Gligoric, L. Eloussi, and D. Marinov. Practical regression test selection with dynamic file dependencies. In *International Symposium on Software Testing and Analysis*, pages 211-222, 2015.
  - [23] Emilio, C., Breno, M., R., V., & Antonia, B. (2019) Scalable Approaches for Test Suite Reduction, *International Conference on Software Engineering*: 419-429.
  - [24] Alessandro, M., Giuseppe, S., & Angelo, S. (2019) Combining Code and Requirements Coverage with Execution Cost for Test Suite Reduction., *IEEE Transactions on Software Engineering*, 45.4: 363-390.
  - [25] Yizhen, C., & Mei-Hwa, C. (2021) Multi-Objective Regression Test Selection, *International Conference on Software Engineering*, 76: 105-116.
  - [26] Hua, Z., Lingming, Z., & Sarfraz, K. (2019) TestSage: Regression Test Selection for Large-Scale Web Service Testing, *International Conference on Software Testing, Verification, and Validation*: 430-440.
  - [27] Nguyen, H. A., Nguyen, T. N., Dig, D., Nguyen, S., Tran, H., & Hilton, M. (2019, May). Graph-based mining of in-the-wild, fine-grained, semantic code change patterns. In 2019 *IEEE/ACM 41st International Conference on Software Engineering (ICSE)* (pp. 819-830). IEEE.
  - [28] D. Kawrykow and M. P. Robillard, "Non-essential changes in version histories," 2011 33rd *International Conference on Software Engineering (ICSE)*, Honolulu, HI, USA, 2011, pp. 351-360, doi: 10.1145/1985793.1985842.
  - [29] D. Silva, M.T. Valente, Refdiff: detecting refactorings in version histories, in: *Proceedings of the 14th International Conference on Mining Software Repositories*, IEEE Press, 2017, pp. 269-279.
  - [30] M. Martinez, M. Monperrus, Coming: a tool for mining change pattern instances from git commits, in: *Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings*, 2019, pp. 79-82.
  - [31] Koyuncu, A., Liu, K., Bissyandé, T. F., Kim, D., Klein, J., Monperrus, M., & Le Traon, Y. (2020). Fixminer: Mining relevant fix patterns for automated program repair. *Empirical Software Engineering*, 25, 1980-2024.
  - [32] Hindle, A., German, D. M., Godfrey, M. W., & Holt, R. C. (2009, May). Automatic classification of large changes into maintenance categories. In 2009 *IEEE 17th International Conference on Program Comprehension* (pp. 30-39). IEEE.
  - [33] S. Gharbi, M.W. Mkaouer, I. Jenhani, M. Ben Messaoud, On the classification of software change messages using multi-label active learning, in: *Proceedings of the 34rd Annual ACM Symposium on Applied Computing (SAC)*, 2019, pp. 1760-1767.
  - [34] Janke, M., & Mäder, P. (2020). Graph based mining of code change patterns from version control commits. *IEEE Transactions on Software Engineering*, 48(3), 848-863.
  - [35] Liu, Xiaojia, and Pengju Wei. "Test cases selection method of rapid regression testing." 2022 *IEEE 10th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*. Vol. 10. IEEE, 2022.
  - [36] Shantanu, S., Rajesh, K., Sriram, P., & Shwetha, B. (2020) Regression Test cases selection using Natural Language Processing, 2020 *International Conference on Intelligent Engineering and Management (ICIEM)*: 301-305.